1.State the role of digital circuit technology in modern communication systems.

Answer : Role of Digital Circuit Technology in Modern Communication Systems:

1. Data Processing and Encoding: Digital circuits (like microprocessors and DSPs) are used to convert analog signals (like voice or video) into digital data (0s and 1s) and compress it for efficient transmission.
2. Error Detection and Correction: They include special circuits (like encoders and decoders) that detect and fix errors caused by noise during transmission, ensuring the data is received accurately.
3. Modulation and Demodulation: Digital circuits perform modulation (changing the signal to travel over air/cables) and demodulation (converting it back) in modern modems and routers to enable fast internet and wireless communication.
4. Switching and Routing: They are the backbone of network switches and routers, which direct data packets to the correct destination, enabling complex networks like the internet.
5. Security and Encryption: Digital circuits implement encryption algorithms to scramble data, ensuring privacy and security during communication (e.g., in mobile phones and online banking).

2.Compare analog and digital circuits,highlighting their advantages,disadvantages and practical applications.

Answer : Comparison of Analog and Digital Circuits:

Feature	Analog Circuits	Digital Circuits
Signal Type	Process continuous signals (varying voltage/current, e.g.,sine waves).	Process discrete signals (binary 0s and 1s or voltage levels).
Accuracy	Prone to errors due to noise and component aging;accuracy degrades over time.	Highly accurate;noise is eliminated because signals are regenerated as pure 0/1.
Design Complexity	Complex to design; requires deep knowledge of physics and mathematics.	Easier to design; uses standard logic gates and modules; relies on Boolean algebra.
Flexibility	Difficult to change; requires changing physical	Highly flexible; functions can be changed by

	components (resistors, capacitors).	modifying software/code (e.g., in FPGAs/microcontrollers).
Power Consumption	Generally consumes higherpower, especially in linear amplification.	Consumes lower power (CMOS technology); power is used only during switching.
Data Storage	Cannot store data easily; uses capacitors (temporary).	Can store data indefinitely using memory devices (RAM, ROM, flip-flops)

Advantages & Disadvantages:

	Analog Circuits	Digital Circuits
Advantages	• Better for capturing real-world signals (natural form). • No need for analog-to-digital conversion (simpler in some cases). • Higher speed for certain simple operations.	• High noise immunity and reliable. • Easy to reproduce and manufacture (mass production). • Programmable and upgradable without hardware changes.
Disadvantages	• Highly affected by noise and distortion. • Hard to troubleshoot and design. • Expensive to redesign for new features.	• Requires ADC (Analog-to-Digital) and DAC (Digital-to-Analog) converters to interface with the real world. • Can have latency due to clock/synchronization. • Uses more chip area for complex functions.

Practical Applications:

Analog Circuits	Digital Circuits
• Audio amplifiers(speakers,microphones). • Radio tuners and FM receivers. • Power supplies(voltage regulators) • Sensor interfaces(temperature,pressure)	• Computers and laptops (CPU, memory). • Smartphones and digital watches. • Internet routers and modems. • Digital cameras, LEDs, and display screens.

3.Analyze the impact of logic minimization on circuit cost,speed and power consumption.
Answer:
Logic minimization is the process of simplifying a Boolean expression so that a digital circuit uses fewer logic gates and connections while producing the same output.

Factor	Impact of Logic Minimization
Circuit Cost	Decreases - fewer gates, wires, and hardware components are required, reducing manufacturing cost and circuit size.
Speed	Increases - a simplified circuit usually has fewer logic levels, so the signal travels through fewer gates. This reduces propagation delay and makes the circuit faster.
Power Consumption	Decreases - fewer gates and switching nodes mean less dynamic power is consumed. Smaller circuits also generally have lower capacitance.

Example-
Suppose a Boolean function is: $Y = AB + AB'$
Using Boolean algebra: $Y = A(B + B')$
Since: $B + B' = 1$
Therefore: $Y = A$

Before minimization:
Requires an AND gates + NOT gate + OR gate.

After minimization:
Only a direct connection/buffer for A is needed.

So, minimization results in:

Lower cost → fewer gates/components

Higher speed → less propagation delay

Lower power → fewer switching elements

4.Is minimizing the number of logic gates always the best optimization strategy?Explain with examples.

Answer:

No, minimizing the number of logic gates is not always the best optimization strategy. While it reduces silicon area and manufacturing cost, modern digital design prioritizes a trade-off between area, speed, power, and testability.

The four key reasons are:

1. Speed (Propagation Delay):
    - Minimized logic often uses complex gates (e.g., a single 8-input AND gate) or long cascaded chains.
    - Example: A balanced tree of smaller 2-input NAND gates uses *more* gates but reduces the logic depth from 8 levels to 3 levels, making the circuit run significantly faster.
2. Power Consumption (Glitching):
    - Minimized expressions often have shared internal nodes that cause unwanted transient toggles (glitches) when inputs arrive at different times.
    - Example: The factored form $A \cdot (B+C)$ uses fewer gates than $AB+AC$,but the factored form can produce a glitch at the output, wasting dynamic power. The larger, un-factored form is more power-efficient.
3. Wiring Dominance (Interconnects):
    - In deep-submicron technologies (e.g., 5nm), wire resistance and capacitance dominate delay and power more than transistors do.

- Example: A ripple-carry adder has minimal gates but requires long carry chains across the chip. A Kogge-Stone adder uses *more* gates but has short, regular, parallel wires, resulting in much higher clock speeds.

4. Testability and Yield:
    - Minimized circuits can hide internal faults, making them untestable during manufacturing.
    - Example: Designers deliberately add extra redundant gates (scan chains) to observe internal nodes. Although this increases gate count, it ensures defective chips are caught, improving overall manufacturing yield.

5.Design a logic circuit with inputs P,Q,R so that output is high whenever P is 0 or whenever Q=R=1.Use NAND gates only.

Answer:

Three inputs signals:P,Q and R; One output signal :Y

Output: Y=1(HIGH) ; if P=0 or Q=R=1

=0(LOW) ; otherwise.

We now develop the truth table below:

Truth table

Input combination			Output,Y
P	Q	R	
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0

1	1	0	0
1	1	1	1

Now write the output expression in SOP form.Consider the input combinations (minterms) for which Y=1. In this case,Y will be in Sum-of-Products form as follows:

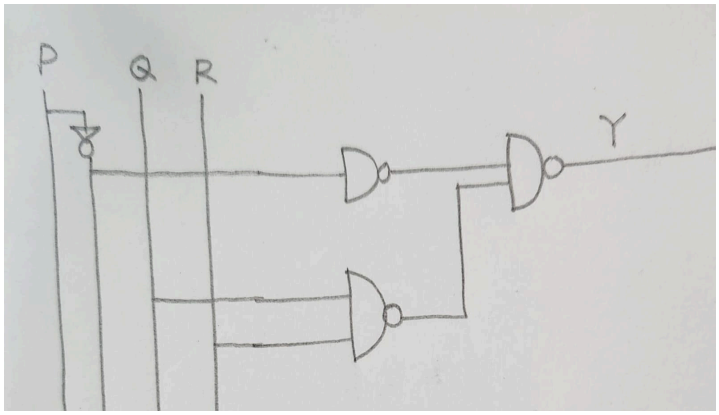
$$Y = P'Q'R' + P'Q'R + P'QR' + P'QR + PQR$$

Simplify the output expression using K-map in SOP form:

P\QR	00	01	11	10
0	1	1	1	1
1	0	0	1	0

The simplified expression in SOP form is $Y = P' + QR$

Now implement the simplified expression:



6.Design a logic circuit that has four inputs (A,B,C and D) and an output (Y) ,such that,the output is a 1 whenever the decimal equivalent of $(ABCD)_2$ is divisible by either 3 or 5.

Answer:

Four input signals:A,B,C and D; One output signal: Y

Output: Y= 1 (HIGH) when decimal value of $(ABCD)_2$ is divisible by 3 OR 5

=0(LOW) otherwise

We now develop the truth table below:

Truth table

Input combination				Output,Y
A	B	C	D	
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

Now write the output expression in SOP form.Consider the input combinations (minterms) for which Y=1. In this case,Y will be in Sum-of-Products form as follows:

$$Y = A'B'C'D' + A'B'CD + A'BC'D + A'BCD' + AB'C'D + AB'CD' + ABC'D' + ABCD$$

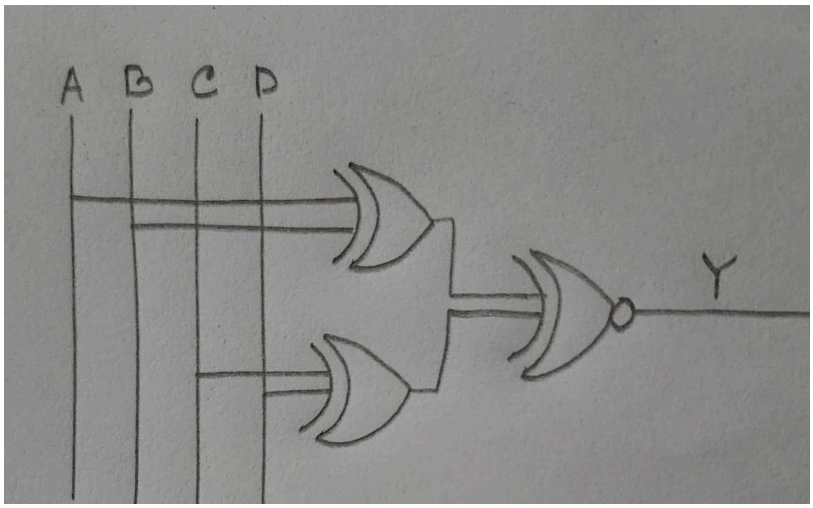
Simplify the output expression using K-map in SOP form:

AB\CD	00	01	11	10
00	1	0	1	0
01	0	1	0	1
11	1	0	1	0
10	0	1	0	1

The simplified expression in SOP form is:$Y = (A \oplus B)'(C \oplus D)' + (A \oplus B)(C \oplus D)$

$=(A \oplus B) \odot (C \oplus D)$

Now implement the simplified expression:



7.Design a logic circuit that accepts a 3-bit number and generates an output binary number equal to the square of the input number.

<u>Answer</u>:

Let,Input:A,B,C

Since,the maximum 3-bit number is 7,its square is:$7^2 = 49 = (110001)_2$

Therefore, the output requires 6 bits, denoted by:$Y_5 Y_4 Y_3 Y_2 Y_1 Y_0$

Truth table:

Input			Decimal	Decimal	Outputs					
A	B	C		Square	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	0
0	0	1	1	1	0	0	0	0	0	1
0	1	0	2	4	0	0	0	1	0	0
0	1	1	3	9	0	0	1	0	0	1
1	0	0	4	16	0	1	0	0	0	0
1	0	1	5	25	0	1	1	0	0	1
1	1	0	6	36	1	0	0	1	0	0
1	1	1	7	49	1	1	0	0	0	1

Simplified Boolean Expressions from the truth table:

A\BC	00	01	11	10
0	0	1	1	0
1	0	1	1	0

$Y_0 = C$

A\BC	00	01	11	10
0	0	0	0	0
1	0	0	0	0

$Y_1 = 0$

A\BC	00	01	11	10
0	0	0	0	1
1	0	0	0	1

$Y_2 = BC'$

A\BC	00	01	11	10
0	0	0	1	0
1	0	1	0	0

$Y_3 = C(A \oplus B)$

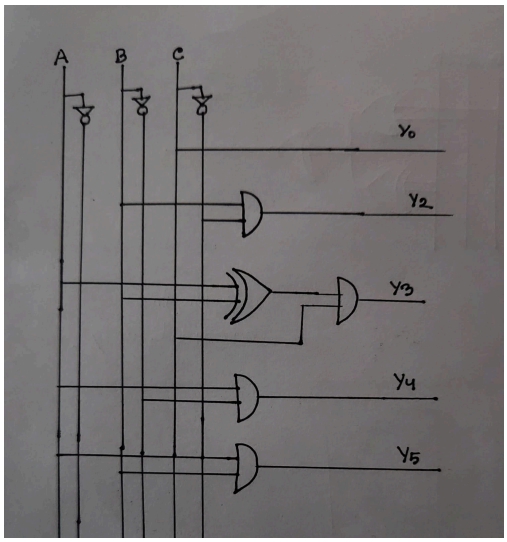
A\BC	00	01	11	10
0	0	0	0	0
1	1	1	1	0

$Y_4 = AB'$

A\BC	00	01	11	10
0	0	0	0	0
1	0	0	1	1

$Y_5 = AB$

Now implement the simplified expression:



8.Design a logic circuit that accepts a 4-bit number and generates the 2's complement of the input number.

Answer:

The 2's complement of an N-bit binary number is obtained by:

- Inverting all bit's(1's complement)
- Adding 1 to the result

Let's 4 Input:A,B,C,D

4 Output:Y_3, Y_2, Y_1, Y_0

Truth table:

A	B	C	D	Input (Decimal)	2's comp(Decimal)	Y_3	Y_2	Y_1	Y_1
0	0	0	0	0	0	0	0	0	0
0	0	0	1	1	15	1	1	1	1
0	0	1	0	2	14	1	1	1	0
0	0	1	1	3	13	1	1	0	1
0	1	0	0	4	12	1	1	0	0
0	1	0	1	5	11	1	0	1	1
0	1	1	0	6	10	1	0	1	0
0	1	1	1	7	9	1	0	0	1
1	0	0	0	8	8	1	0	0	0
1	0	0	1	9	7	0	1	1	1
1	0	1	0	10	6	0	1	1	0
1	0	1	1	11	5	0	1	0	1
1	1	0	0	12	4	0	1	0	0
1	1	0	1	13	3	0	0	1	1
1	1	1	0	14	2	0	0	1	0
1	1	1	1	15	1	0	0	0	1

Simplificed Boolean Expressions:

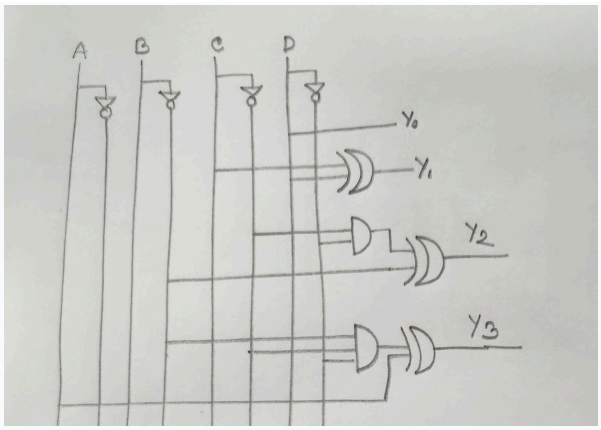
$Y_0 = D$

$Y_1 = C \oplus D$

$Y_2 = B' \oplus (C'D')$

$Y_3 = A' \oplus (B'C'D')$

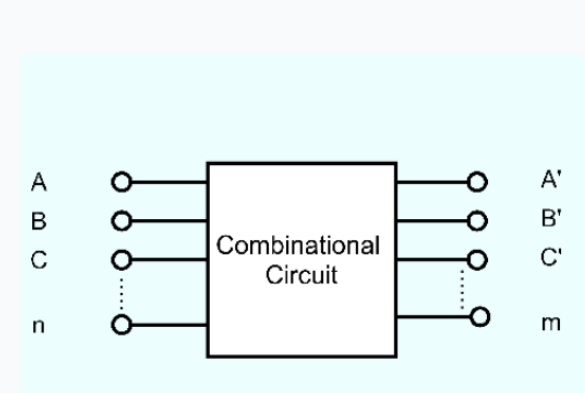
Now implement the simplified expression:

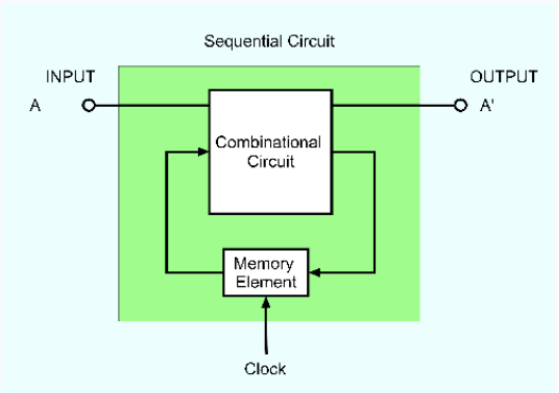


9.Compare combinational and sequential logic circuits with respect to design,operation and applications.

Answer:

Combinational Circuits	Sequential Circuits
Output depends only on current inputs	Output depends on current inputs+ previous state (history)
No memory elements	Has memory elements (flip-flops)
No feedback path	Has feedback path





Operation:

Combinational Circuits	Sequential Circuits
Immediate response - output changes as soon as input changes (after propagation delay)	Clock-dependent response - output changes only at clock edges
tateless - no memory of past inputs	Stateful - remembers previous inputs/states
Generally faster operation	Generally slower due to clock constraint

Applications:

Combinational Circuits	Sequential Circuits
● Adders, Subtractors ● Multiplexers, Demultiplexers ● Encoders, Decoders ● Comparators ● Code converters (Binary to BCD, 7-segment decoder)	● Counters ● Registers, Shift registers ● Finite State Machines (FSM) ● Memory units ● Traffic light controllers, Sequence detectors

10.Design and explain the circuit to add the following bits 1011 and 1100 using block - diagram.

Answer:

To add two 4-bit binary numbers, 1011 and 1100, we use a 4-bit parallel binary adder. It can be implemented using four Full Adders (FA) connected in cascade.

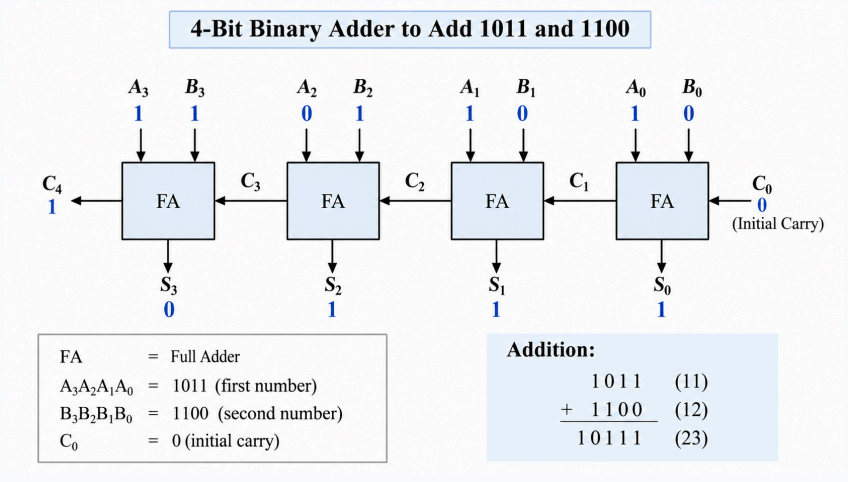
Given,A=1011

B=1100

Initial carry:$C_0 = 0$

Block Diagram:

The four Full Adders are connected from the LSB to the MSB:



Thus, the final output is:$C_4S_3S_2S_1S_0 = 10111$

11.A standard 8-to-3 encoder receives two active inputs simultaneously.What output will it produce?Why is this a problem?

Inputs	Y_2	Y_1	Y_0
I_0	0	0	0
I_1	0	0	1
I_2	0	1	0
I_3	0	1	1
I_4	1	0	0
I_5	1	0	1
I_6	1	1	0
I_7	1	1	1

$Y_0 = I_1 + I_3 + I_5 + I_7$

$Y_1 = I_2 + I_3 + I_6 + I_7$

$Y_2 = I_4 + I_5 + I_6 + I_7$

When two inputs are active simultaneously in a standard 8-to-3 encoder, the output will be the bitwise OR of the binary codes of both active inputs.

Example:If $I_3(011)\ and\ I_5(101)$ are active:

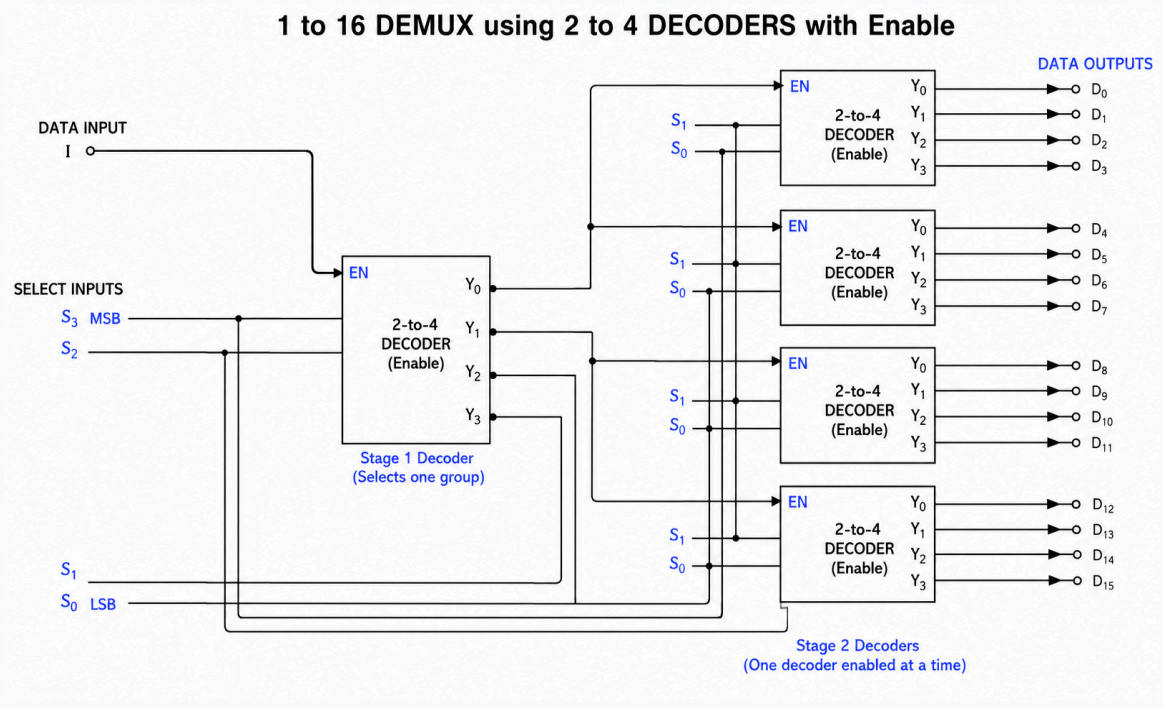
Output Bit	Expression	Value
$Y_0 = I_1 + I_3 + I_5 + I_7$	0+1+1+0	1
$Y_1 = I_2 + I_3 + I_6 + I_7$	0+1+0+0	1
$Y_2 = I_4 + I_5 + I_6 + I_7$	0+1+0+0	1

Output = 111 (which is binary 7) - an incorrect output.

Problem	Explanation
Ambiguous Output	The output 111 doesn't represent either I_3 (011) or I_5 (101) → Meaningless result
Loss of Information	Same output (111) is produced for I_7 alone, so we can't distinguish between I_3+I_5 and I_7
Data Corruption	In real applications (keyboards, interrupt controllers), this causes wrong codes/addresses

12.Implement a 1-to-16 demultiplexer using only 2-to-4 decoders with enable inputs and no other logic gates.Clearly label all inputs,pins and outputs of your circuit.

Answer:



13.Design a logic circuit that compares two 4-bit numbers A and B,to check if they are equal.The circuit has one output x,so that x=1 if A=B and x=0 if A≠ B.

Answer:

Let the two 4-bit numbers be:

$A = A_3A_2A_1A_0$

$B = B_3B_2B_1B_0$

The circuit has one output X such that:

X = $1, A = B$

$X = 0, A \neq B$

For A and B to be equal, all four corresponding bits must be equal:

$A_3 = B_3$, $A_2 = B_2$, $A_1 = B_1$, $A_0 = B_0$

We can compare each pair using an XNOR gate.

XNOR gives:

- Same bits→ 1
- Different bits→ 0

Boolean Expression

For each bit:

$E_3 = A_3 \odot B_3$

$E_2 = A_2 \odot B_2$

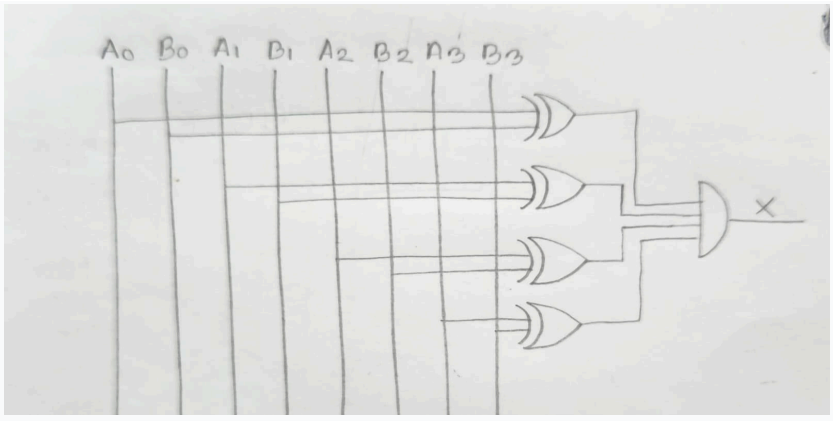
$E_1 = A_1 \odot B_1$

$E_0 = A_0 \odot B_0$

Then, $X = E_3 E_2 E_1 E_0$

Therefore, $X = (A_3 \odot B_3)(A_2 \odot B_2)(A_1 \odot B_1)(A_0 \odot B_0)$

Circuit Implementation:



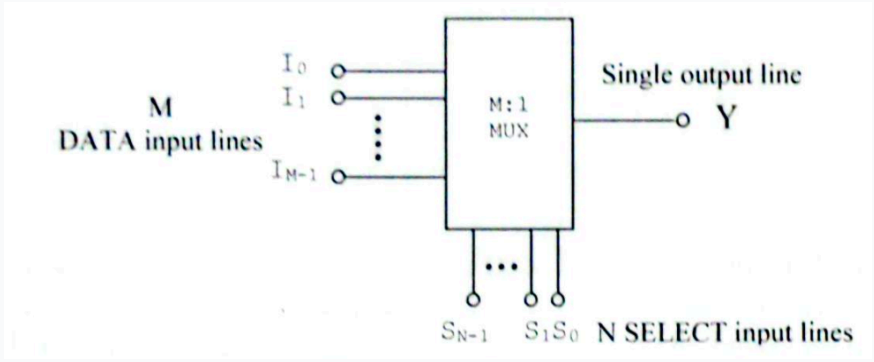
14.Explain how a multiplexer can be used as a universal logic circuit.

Answer:

A multiplexer is a combinational circuit that:

- Has multiple inputs (2^n) and one output
- Selects one input to pass to the output based on n select lines
- Acts like a data selector or rotary switch

Structure:

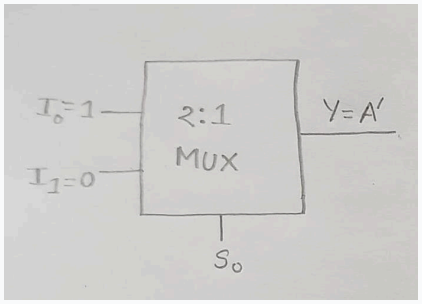


A multiplexer is considered universal because:

1. Any Boolean function can be implemented using only multiplexers
2. It can replace AND, OR, NOT, NAND, NOR, XOR gates
3. It serves as a complete logic family by itself
4. Can implement combinational circuits without using basic gates

NOT Gate using 2:1 MUX

select(s)	I_0	I_1	Output(Y)
S=A	1	0	Y=A’

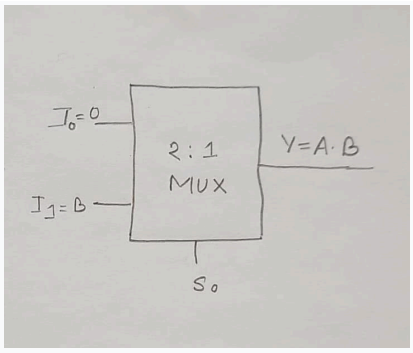


Working:

- When A = 0 → S = 0 → Y = I_0 = 1 = A'
- When A = 1 → S = 1 → Y = I_1 = 0 = A'

AND Gate using 2:1 MUX

select(s)	I_0	I_1	Output(Y)
S=A	0	B	Y=A.B



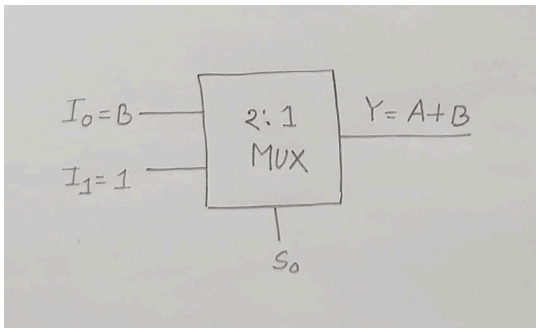
Working:

- When A = 0 → Y = I_0 = 0 = A·B
- When A = 1 → Y = I_1 = B = A·B

OR Gate using 2:1 MUX

select(s)	I_0	I_1	Output(Y)

S=A	B	1	Y = A+B



Working:

- When A = 0 ⟶ Y = I_0 = B = A+B
- When A = 1 ⟶ Y = I_1 = 1 = A+B

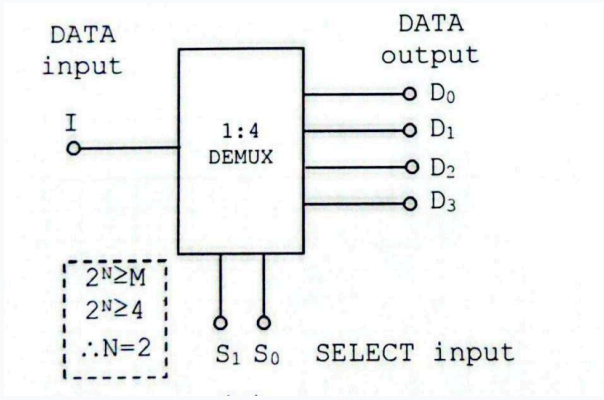
15.Design a circuit to distribute a serial input to four different outputs using a DEMUX.

Answer:

A Demultiplexer (DEMUX) is a combinational circuit that takes one data input and routes it to one of several outputs according to the values of the select inputs.

For distributing one serial input to four different outputs, we use a 1-to-4 DEMUX.

Circuit Diagram:



Truth Table:

SELECT Input		Output			
S_1	S_0	D_0	D_1	D_2	D_3
0	0	I	0	0	0
0	1	0	I	0	0
1	0	0	0	I	0
1	1	0	0	0	I

From the above truth table, the logic expressions for outputs are as follows:

$$D_0 = \overline{S}_1 \overline{S}_0 I,\ D_1 = \overline{S}_1 S_0 I,\ D_2 = S_1 \overline{S}_0 I,\ \text{and}\ D_3 = S_1 S_0 I$$

16.Give a real-life application where a priority encoder is preferred over a standard encoder.

Answer:

A priority encoder is preferred over a standard encoder when more than one input may become active at the same time and the system must decide which input has higher priority.

Example: Interrupt handling in a computer system

In a computer, several devices such as the keyboard, mouse, printer, and network card may request the CPU's attention at the same time.

A priority encoder can assign a priority to each interrupt:

- Highest priority → Critical device
- Medium priority → Network device
- Lower priority → Keyboard
- Lowest priority → Printer

If multiple devices send interrupt requests simultaneously, the priority encoder selects the highest-priority request and sends its binary code to the CPU.

A standard encoder assumes that only one input is active at a time. If multiple inputs are active simultaneously, its output may be ambiguous or invalid.

Therefore:

> Priority encoders are preferred in interrupt handling systems because they can correctly handle multiple simultaneous inputs by selecting the highest-priority one.

17.Compare a decoder with a demultiplexer.Under what conditions can a decoder function as a demultiplexer?

Answer:

Both a decoder and a demultiplexer (DEMUX) use select inputs to activate one of several output lines, but their purposes are different.

Feature	Decoder	Demultiplexer
Main function	Converts binary input into one active output	Routes one data input to one selected output
Data input	Usually has n input lines	Has 1 data input
Select inputs	n input bits act as selection	n select lines
Number of outputs	2^n	2^n
Data routing	Does not route data	Routes data
Example	2-to-4 decoder	1-to-4 DEMUX

A decoder can function as a demultiplexer when one of its inputs is used as the data input (enable input), while the remaining inputs are used as select lines.

For example, consider a 2-to-4 decoder with an Enable (E) input:

- E→Data input
- S_1, S_0→Select inputs
- Y_0, Y_1, Y_2, Y_3 →Outputs

The outputs can be represented as:

$Y_0 = E(S_1S_0)'$

$Y_1 = ES_1'S_0$

$Y_2 = ES_1S_0'$

$Y_3 = ES_1S_0$

These are exactly the same expressions as a 1-to-4 DEMUX.

18.A logic circuit is defined by the following functions:

$f_1 = A'B' + ABC'$

$f_2 = A' + B$

$f_3 = AB + A'B'$

Draw the circuit with a decoder and external gates.

Answer:

The functions have 3 inputs (A, B, C), so we need a 3-to-8 decoder.

A 3-to-8 decoder produces 8 minterms:

Inputs			Outputs
A	B	C	
0	0	0	$D_0 = A'B'C'$
0	0	1	$D_1 = A'B'C$
0	1	0	$D_2 = A'BC'$
0	1	1	$D_3 = A'BC$

1	0	0	$D_4 = AB'C'$
1	0	1	$D_5 = AB'C$
1	1	0	$D_6 = ABC'$
1	1	1	$D_7 = ABC$

Expand Each Function into Minterms

$$f_1 = A'B' + ABC' = A'B'(C + C') + ABC' = A'B'C + A'B'C' + ABC' = D_0 + D_1 + D_6$$

$$f_2 = A' + B = A'(B + B')(C + C') + B(A + A')(C + C')$$

$$= A'BC + A'BC' + A'B'C + A'B'C' + ABC + ABC' + A'BC + A'BC'$$

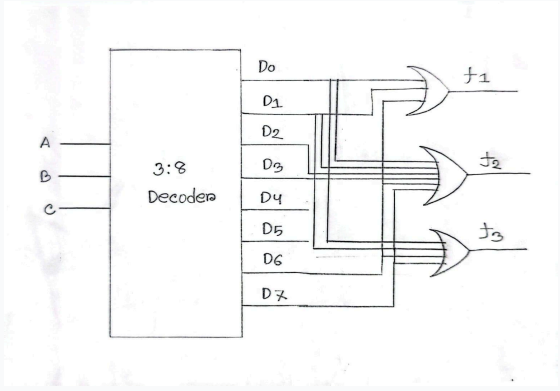
$$= D_0 + D_1 + D_2 + D_3 + D_6 + D_7$$

$$f_3 = AB + A'B'$$

$$= AB(C + C') + A'B'(C + C')$$

$$= D_0 + D_1 + D_6 + D_7$$

Circuit Diagram:



19.Under what conditions would you choose a D flip-flop instead of a JK flip-flop?Justify your answer.

Answer:

A D flip-flop should be chosen over a JK flip-flop when the main requirement is to store or transfer a single bit of data and a simple circuit is preferred.

Conditions

We choose a D flip-flop when:

1. Data storage is the main purpose
   A D flip-flop directly stores the value of the D input at the active clock edge.
2. A simple circuit is required
   A D flip-flop has only one data input, D, making the circuit simpler than a JK flip-flop.
3. No toggle or reset/set behavior is required through the data inputs
   JK flip-flops are useful when we need set, reset, hold, and toggle operations. If these features are unnecessary, D flip-flop is more appropriate.
4. Registers or data transfer circuits are being designed
   D flip-flops are commonly used in registers, shift registers, and data storage systems.

Justification

For a D flip-flop:

Q_{next}=D

So, whatever value is present at D is transferred directly to Q on the active clock edge.

For a JK flip-flop, two inputs J and K must be considered, and it provides additional operations such as toggling:

$J = K = 1 \Rightarrow Q_{next} = Q'$

Therefore, using a JK flip-flop when only simple data storage is needed would add unnecessary complexity.

20.Compare SR,JK,D and T flip-flops in terms of functionality,complexity and typical applications.

Answer:

Feature	SR Flip-Flop	JK Flip-Flop	D Flip-Flop	T Flip-Flop
Inputs	S, R	J, K	D	T
Main Function	Set and Reset	Set, Reset and Toggle	Data storage	Toggle
Hold	S=0, R=0	J=0, K=0	D retains previous value	T=0
Set	S=1, R=0	J=1, K=0	D=1	Not directly
Reset	S=0, R=1	J=0, K=1	D=0	Not directly
Toggle	Not available	J=K=1	Not directly	T=1
Invalid condition	S=R=1	No invalid condition	None	None
Complexity	Simple	More complex	Simple	Simple
Typical Applications	Basic memory circuits, Control circuits ,Switch debouncing	Counters, Frequency division, Sequential control circuits	Registers, Shift registers, Data storage, Data transfer circuits	Binary counters, Frequency dividers, Toggle circuits

21.Compare edge-triggereed and level-triggered flip-flops.Which is more suitable for high-speed applications and why?

Answer:

Comparison Between Edge-Triggered and Level-Triggered Flip-Flops:

Feature	Edge-Triggered	Level-Triggered

Operation	Changes only at a clock edge	Changes during a particular clock level
Triggering	Rising edge (↑) or falling edge (↓)	HIGH level or LOW level
Input response	Responds at one instant	Responds throughout the active clock level
Speed	Suitable for high-speed systems	Less suitable for very high-speed systems
Timing control	More precise	Less precise
Race-through problem	Generally avoided	Can occur in some designs
Complexity	Usually more complex	Usually simpler
Applications	High-speed processors, registers, synchronous systems	Simple control circuits, latches, low-speed systems

Edge-triggered flip-flop is more suitable for high-speed applications.Reason:It samples the input only at a specific clock edge. This provides better timing control, prevents unwanted changes during the clock period, and reduces race-through problems, making it more suitable for high-speed synchronous digital systems.

22. Is a JK flip-flop always a better choice because it is more versatile?Defend your answer.

Answer:

No. A JK flip-flop is not always the better choice, even though it is more versatile.

A JK flip-flop can perform hold, set, reset, and toggle operations:

J	K	Operation
0	0	Hold
0	1	Reset
1	0	Set

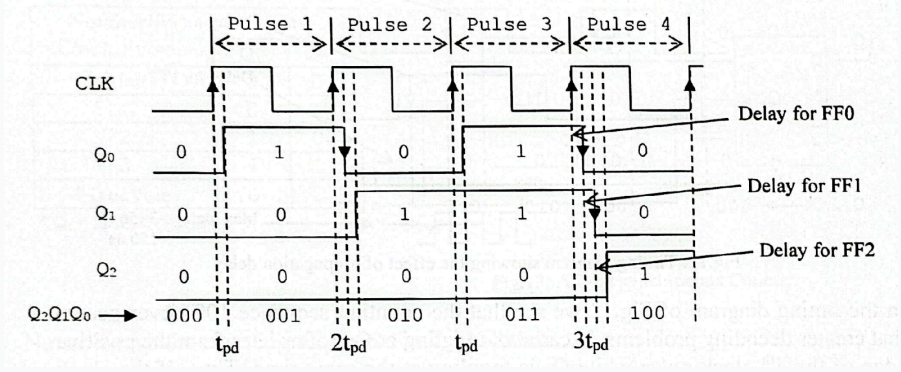
1	1	Toggle

Because of this versatility, it can perform the functions of other flip-flops with suitable connections. However, more functionality does not always mean a better choice.

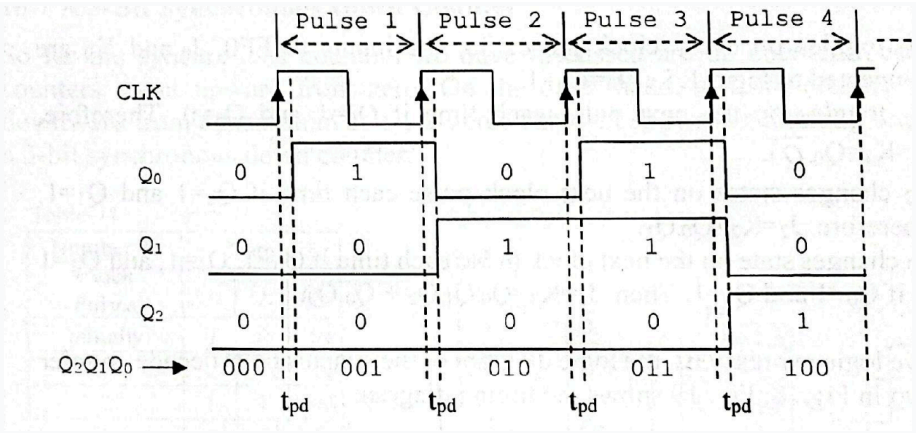
23.Under what circumstances would you choose a synchronous design over an asynchronous design?Justify your answer.

Answer:

A synchronous design is preferred when predictable timing, reliable operation and easy coordination between circuit components are important. Although asynchronous designs can respond faster because they do not wait for a clock, they are more sensitive to propagation delays, race conditions, and timing problems.



In an asynchronous design, the flip-flops do not receive the clock simultaneously. The output of one flip-flop triggers the next flip-flop. Therefore, propagation delays accumulate from one stage to another.



In a synchronous design, all flip-flops receive the same clock signal simultaneously. Therefore, the circuit has better timing control and less accumulated propagation delay.

So, For high-speed applications, synchronous design is preferred because all flip-flops are triggered by the same clock, reducing cumulative propagation delay and providing more predictable timing.

24.Under what circumstances would an asynchronous counter be a better choice than a synchronous counter?

Answer:

An asynchronous counter (ripple counter) can be a better choice when simplicity, low hardware cost, and low-speed operation are more important than high speed.

Circumstances

1. Low-speed applications
   When the counting frequency is low, the propagation delay is usually not a major problem.
2. Simple circuit is required
   In an asynchronous counter, only the first flip-flop receives the external clock. The remaining flip-flops are triggered by the outputs of previous flip-flops, so the circuit is relatively simple.
3. Low hardware cost is desired
   It requires less additional logic, making it cheaper and easier to implement.
4. Small number of counting stages
   For a small number of flip-flops, the accumulated propagation delay may be acceptable.
5. Simple frequency division is needed
   Asynchronous counters are commonly used as frequency dividers.

In an asynchronous counter, the clock does not reach all flip-flops simultaneously. Therefore, a propagation delay occurs from one flip-flop to the next:

$$t_{total} \approx n \times t_{pd}$$

where n is the number of flip-flops.

For high-speed applications, this delay becomes a problem. But for low-speed and simple applications, the advantages of lower complexity and cost can make an asynchronous counter preferable.

Example

An asynchronous counter can be used in a simple digital frequency divider or a low-speed event counter where very high counting speed is not required.

25.Why are synchronous counters preferred in modern high-speed processors?

Answer:

Synchronous counters are preferred in modern high-speed processors because all flip-flops are triggered by the same clock signal. This allows the counter to operate faster and with more predictable timing.

Main Reasons

1. Less propagation delay
   In a synchronous counter, all flip-flops receive the clock simultaneously. Therefore, the delay does not accumulate from one flip-flop to the next.
2. Higher operating speed
   Since cumulative ripple delay is avoided, synchronous counters can operate at much higher frequencies.
3. Better timing control
   All state changes occur at controlled clock edges, which makes timing more predictable.
4. Suitable for complex processors
   Modern CPUs contain many interconnected sequential circuits. Synchronous operation makes it easier to coordinate these circuits.
5. Reliable operation
   Synchronous counters reduce unwanted intermediate states and timing problems that can occur in asynchronous counters.